LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 4
Student Name: B.Manoj
Roll No.: A24126552072
Date: 30-08-2026

---

1. TITLE

Design and Implementation of Interactive Todo List Application Using JavaScript DOM Manipulation

2. AIM

To design and implement an interactive Todo List web application using JavaScript that allows users to add tasks dynamically, toggle task completion with strikethrough styling, and remove individual tasks from the DOM.

3. OBJECTIVE

The objectives of this experiment are:

- To understand dynamic list manipulation in JavaScript.
- To capture and trim user input from text boxes.
- To create and insert list items (<li>) with child elements dynamically.
- To implement class toggling (classList.toggle) on user click for task completion.
- To remove parent DOM nodes dynamically using parentElement.remove().

4. THEORY

Dynamic DOM manipulation enables web applications to respond to user actions without reloading the page. Tasks can be created as list items, appended to an unordered list container, styled conditionally with CSS classes, and removed from the DOM hierarchy upon user interaction.

Application Flow:
User Types Task -> Click Add -> Create <li> Element -> Click Text (Toggle Done) -> Click Delete (Remove Node)

5. ALGORITHM / PROCEDURE

1. Design a centered Todo container box with a text input field, 'Add' button, and an unordered list <ul id='list'>.
2. Define CSS styles for task items, completed tasks (.done with line-through), and delete buttons.
3. Implement the addTask() JavaScript function.
4. Retrieve input value using document.getElementById('task').value.trim().
5. Validate input to ensure empty tasks are ignored.
6. Create a new <li> element and set its innerHTML with a clickable text span and a Delete button.
7. Attach onclick handlers for classList.toggle('done') on the span and parentElement.remove() on the button.
8. Append the <li> to the list container and clear the input field.
9. Test adding, toggling, and deleting tasks.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Todo List</title>
<style>
body{font-family:Arial;background:#eef2ff;display:flex;justify-content:center;padding:50px}
.box{background:white;padding:30px;width:500px;border-radius:15px;box-shadow:0 5px 15px #aaa}
h2{text-align:center;color:#4f46e5;font: size 28px;px}
.row{display:flex;gap:10px}
input{flex:1;padding:15px;font-size:18px;border:1px solid #ccc;border-radius:8px}
button{background:#4f46e5;color:white;border:0;padding:15px 20px;font-size:16px;border-radius:8px;cursor:pointer}
ul{padding:0;list-style:none}
li{background:#f1f5f9;margin:10px 0;padding:15px;font-size:18px;border-radius:8px;
display:flex;justify-content:space-between;align-items:center}
.done{text-decoration:line-through;color:#888}
.del{background:#ef4444;padding:8px 12px;font-size:14px}
</style>
</head>

<body>
<div class="box">
<h2>■ Todo List</h2>

<div class="row">
<input id="task" placeholder="Enter task">
<button onclick="addTask()">Add</button>
</div>

<ul id="list"></ul>
</div>

<script>
function addTask(){
 let input=document.getElementById("task");
 let value=input.value.trim();
 if(!value)return;

 let li=document.createElement("li");
 li.innerHTML=`<span onclick="this.classList.toggle('done')">${value}</span>
 <button class="del" onclick="this.parentElement.remove()">Delete</button>`;

 document.getElementById("list").appendChild(li);
 input.value="";
}
</script>
</body>
</html>
```

7. CODE EXPLANATION

Code / Syntax	Explanation
input.value.trim()	Extracts user input and removes leading/trailing whitespaces.
if(!value) return;	Prevents adding blank or empty task items.
document.createElement('li')	Creates a new list item element in memory.
this.classList.toggle('done')	Toggles the 'done' class on click for line-through text formatting.
this.parentElement.remove()	Removes the parent node directly from the DOM tree on clicking Delete.
document.getElementById('list').appendChild(li)	Appends the new task item to the unordered list.

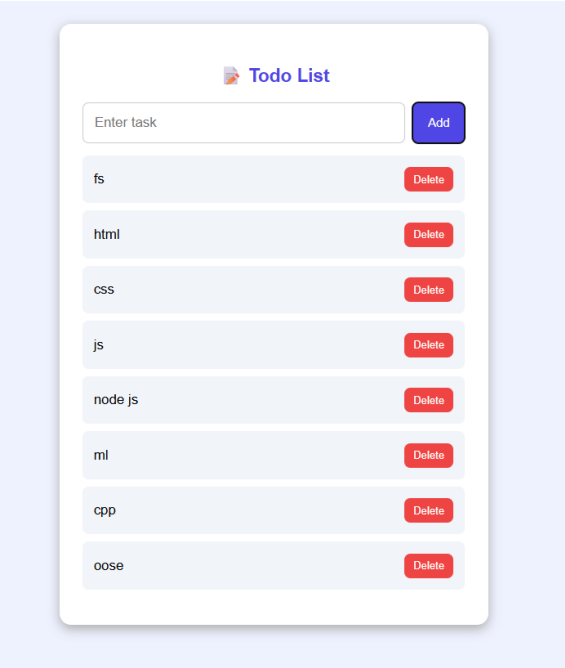
8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Add Task	Enter 'Complete FS Lab' & click Add	New task item appears in the list	**Pass**
TC-02	Empty Input Check	Click Add with empty input	No empty item added to the list	**Pass**
TC-03	Toggle Complete	Click on task text	Task text struck through with line-through style	**Pass**
TC-04	Delete Task	Click Delete button	Task item is completely removed from list	**Pass**
TC-05	Input Reset	Check input box after adding	Input box is cleared ready for next task	**Pass**

9. OUTPUT

Output 1: Interactive Todo List Application with Add and Delete Controls



10. RESULT

Thus, the interactive Todo List application was successfully implemented using JavaScript DOM manipulation and verified for task addition, completion toggling, and task deletion.